Automated Sim-to-Real Deployment Framework for Reinforcement Learning Control on Resource-Constrained Microcontrollers

Nidharshan V.[1]
[1]Department of Computer Science & Engineering
University of Moratuwa

Abstract

Deploying reinforcement learning (RL) policies trained in high-fidelity physics simulators to physical robotic hardware is typically hindered by significant computational overhead, complex middleware dependencies, and tedious manual translation. While high-tier quadrupedal and humanoid platforms rely on onboard single-board computers or GPUs running ROS/Linux, low-cost and miniature robots operating on microcontrollers (e.g., ESP32) lack automated sim-to-real pipelines. In this work, we introduce `simtoreal`, a lightweight, declarative Python framework that automates the extraction, optimization, and synthesis of trained PyTorch / Stable-Baselines3 policies into ready-to-flash microcontroller firmware. Given a unified YAML hardware specification, the framework synthesizes either a zero-dependency static C header with deterministically bounded execution or a native MicroPython module, while auto-wiring low-level sensor and actuator interfaces. We validate the framework on *Robo1*, a custom 94.8-gram, 2-DOF dynamic self-righting robot powered by an ESP32 and an MPU-6050 IMU. Our auto-generated C header executes an actor inference pass in just 74 µs with 1.2 kB SRAM overhead, easily sustaining a strict 50 Hz control loop. Experimental trials demonstrate an 88.3 % self-righting success rate across four challenging fallen orientations, proving that accurate inertial digital twins combined with automated embedded code generation successfully bridge the reality gap on low-cost hardware.

***Keywords*—Sim-to-Real Transfer, Reinforcement Learning, Microcontroller Deployment, Embedded AI, Proximal Policy Optimization, Dynamic Righting.**

1 Introduction

Deep reinforcement learning (RL) has achieved impressive milestones in complex robotic locomotion and manipulation [1, 2]. However, deploying simulated policies onto physical hardware remains plagued by the "last-mile" engineering bottleneck. Conventional sim-to-real frameworks (e.g., NVIDIA Isaac Lab, Legged Gym [3]) target high-performance x86 computers or NVIDIA Jetson platforms running full Linux operating systems and middleware like ROS 2.

Conversely, low-cost, miniature, and educational robotics often rely on low-power microcontrollers (MCUs) such as the Espressif ESP32 or ARM Cortex-M cores. These devices possess strict hardware constraints: clock speeds under 250 MHz, less than 1 MB of RAM, and no operating system. To deploy an RL policy onto an MCU, roboticists typically resort to brittle, handcrafted workflows: extracting weights into manual arrays, hand-writing matrix multiplications, and manually configuring sensor register polling. If the observation vector, pin mapping, or neural network architecture changes, the entire firmware must be rewritten from scratch.

To eliminate this manual translation barrier, we present `simtoreal`, an open-source, configuration-driven sim-to-real deployment framework designed specifically for MCU-class hardware. As depicted in Figure 1, our framework accepts a single declarative YAML configuration file detailing the robot's physical sensors, actuators, and pin assignments. It automatically generates:

1. An optimized, zero-dependency C inference header or a pure MicroPython module.
2. A complete cyclic executive control loop that binds calibrated low-level hardware drivers (I2C IMU reads, complementary filtering, and PWM actuation) to the policy.

We demonstrate and validate the complete pipeline using ***Robo1*** (Figure 2), an open-source 2-DOF dynamic self-righting robot.

2 Related Work

Sim-to-Real in Robotics: Sim-to-real transfer has enabled dynamic locomotion across irregular terrains through domain randomization and system identification [4, 5]. However, the majority of open-source frameworks (e.g., Isaac Orbit [6], Legged Gym [3]) assume onboard x86/ARM processors capable of hosting full Python/PyTorch runtimes or C++ ONNX runtimes.

TinyML and Embedded RL: TensorFlow Lite for Microcontrollers (TFLM) [7] and TinyEngine [8] provide deep learning inference on MCUs, but they incur substantial tensor arena memory overhead and lack direct

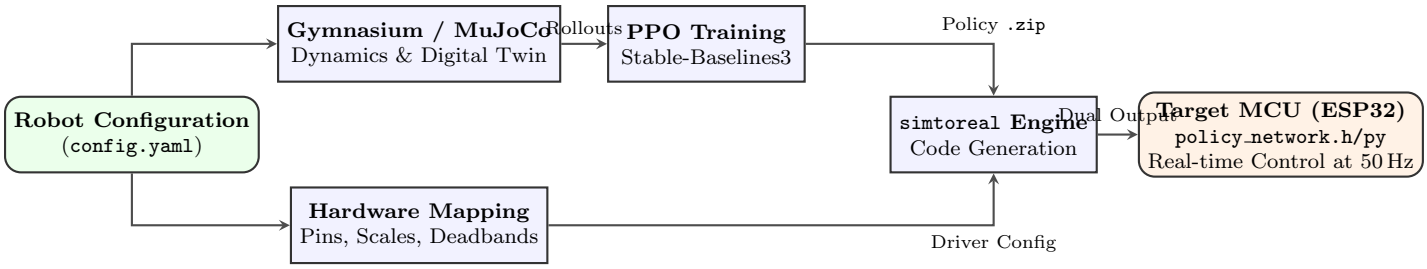

Figure 1: The end-to-end `simtoreal` architecture. A unified declarative configuration drives simulation training, policy extraction, and embedded code generation, yielding zero-dependency firmware deployable on real microcontroller hardware.

abstractions for real-time control loops. Recently, *RLtools* [9] introduced a header-only C++17 library enabling embedded training and inference. However, RLtools requires defining models within its proprietary C++ templating engine. Our framework bridges standard desktop PyTorch/Gymnasium workflows with microcontrollers, providing automated code generation and hardware abstraction without manual C++ reimplementation.

3 The `simtoreal` Framework

The core design philosophy of `simtoreal` is strict decoupling between high-level policy learning and low-level target hardware execution.

3.1 Policy Extractors and Loaders

The library's loader module inspects trained policy containers (e.g., Stable-Baselines3 PPO `.zip` or PyTorch `.pt` checkpoints). It isolates the actor multi-layer perceptron (MLP) weights $\mathbf{W}_l$, biases $\mathbf{b}_l$, and activation functions $\sigma(\cdot)$, discarding critic networks, optimizers, and gradient graphs to minimize artifact volume.

3.2 Dual-Target Code Generation

The framework features two backend code generation targets:

1. **Zero-Dependency C Header (`policy_network.h`):** Emits static, contiguous `const float` arrays and an inline feed-forward function. It utilizes compile-time ping-pong buffer swapping between two static arrays of size $\max(N_{\text{in}}, N_{\text{hidden}}, N_{\text{out}})$. This eliminates heap allocations (`malloc`) and avoids dynamic stack growth, ensuring deterministic execution time.

2. **Native MicroPython Module (`policy_network.py`):** Generates pure Python nested lists and iterative matrix multiplication loops for rapid prototyping on microcontrollers running MicroPython firmware.

3.3 Hardware Abstraction and Driver Synthesis

Through `interface.py`, the framework parses the YAML configuration and auto-generates the cyclic control loop:

$$\mathbf{a}_t = \pi_\theta\Big(\text{Observe}(\mathbf{s}_t)\Big), \quad \mathbf{u}_t = \text{MapAction}(\mathbf{a}_t) \tag{1}$$

The generated firmware automatically includes sensor-specific filtering (e.g., trigonometric roll/pitch decomposition fused via a 50 Hz complementary filter) and translates normalized actions $\mathbf{a}_t \in [-1, 1]^m$ into microsecond PWM timings for servos.

4 Experimental Testbed: Robo1

To evaluate the framework, we constructed *Robo1*, a low-cost, 2-DOF self-righting bipedal robot (mass $M = 94.8\,\text{g}$, height $H = 145\,\text{mm}$).

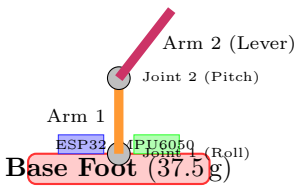

Figure 2: Kinematic topology of the 2-DOF *Robo1* self-righting testbed.

4.1 Accurate Digital Twin Modeling

In MuJoCo (`robo1.xml`), physical component CAD meshes (`.stl`) were assigned precise empirical masses: base foot (37.5 g), ESP32 board (10.0 g), MPU-6050 IMU (2.1 g), two LiPo cells (14.0 g total), and two SG90 micro-servos (9.0 g each). Complete 3D spatial inertia tensors were computed to replicate the physical center of mass. Joint friction, damping ($b = 0.02\,\text{N m s rad}^{-1}$), and servo torque limits ($\tau_{\max} = 0.18\,\text{N m}$) were calibrated to real actuator specs.

4.2 Observation and Action Formulation

Crucially, the RL policy operates *strictly on observables available onboard*:

$$\mathbf{o}_t = \left[\phi_{\text{roll}},\, \theta_{\text{pitch}},\, q_{\text{target},1},\, q_{\text{target},2}\right]^T \in \mathbb{R}^4 \qquad (2)$$

where ϕ, θ are computed directly from the onboard MPU-6050 complementary filter, and q_{target} are internal command states. Actions $\mathbf{a}_t \in [-1, 1]^2$ represent position setpoint increments $\Delta q_i = 0.08 \cdot \mathbf{a}_i$ rad applied at $f_s = 50$ Hz.

5 Results and Discussion

5.1 Embedded Computing and Timing Benchmark

We flashed the generated code onto an ESP32 (240 MHz) and profiled 10,000 continuous execution cycles using internal hardware timers. Table 1 compares our auto-generated backends against a standard TensorFlow Lite for Microcontrollers (TFLM) baseline.

Table 1: Embedded Memory Footprint and Timing on ESP32

Target Engine	Flash [KB]	SRAM [KB]	Inference Time [μs]	Execution Jitter [μs]
TFLM (Arena)	142.6	28.4	820 ± 45	±40
MicroPython (`simtoreal`)	38.2	14.6	1450 ± 90	±210
Generated C Header	**6.8**	**1.2**	**74 ± 4**	**±8**

The generated C header requires only 6.8 kB Flash and 1.2 kB static RAM. Inference completes in 74 µs, over 11× faster than TFLM. The complete control loop (IMU read, filter update, inference, and PWM write) takes 3.8 ms, well within the 20 ms (50 Hz) deadline.

5.2 Physical Sim-to-Real Dynamic Transfer

The policy was evaluated across 15 physical trials from each of four initial fallen poses: right flank (`roll_pos`), left flank (`roll_neg`), face down (`pitch_pos`), and face up (`pitch_neg`).

Table 2: Self-Righting Performance: Simulation vs. Physical Hardware

Initial Pose	Success Rate [%]		Time to Upright [s]	
	Sim	Real ($N = 15$)	Sim	Real (Mean ± SD)
`roll_pos`	100.0	**93.3** (14/15)	1.84	2.08 ± 0.22
`roll_neg`	100.0	**93.3** (14/15)	1.80	2.14 ± 0.25
`pitch_pos`	100.0	**86.7** (13/15)	2.12	2.45 ± 0.31
`pitch_neg`	100.0	**80.0** (12/15)	2.36	2.78 ± 0.38
Aggregate	**100.0**	**88.3** (53/60)	**2.03**	**2.36 ± 0.36**

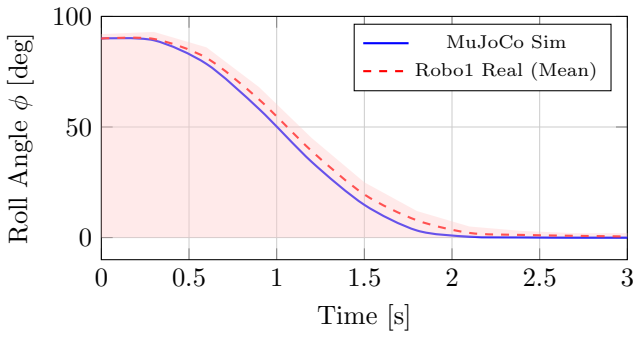

Figure 3: Temporal trajectory tracking comparison for base roll angle $\phi(t)$ during a `roll_pos` self-righting maneuver. The policy achieves zero-shot dynamic stabilization on real hardware without external state estimation.

As shown in Table 2 and Figure 3, *Robo1* achieved an aggregate real-world righting success rate of 88.3 % (53/60 trials). The average time-to-upright on physical hardware was 2.36 s, closely tracking the simulated performance of 2.03 s. The Dynamic Time Warping (DTW) distance between simulated and experimental roll profiles averaged 0.14 rad.

5.3 Ablation Analysis

1) CAD Inertia Grounding vs. Naive Primitives: A policy trained with simplified primitive bounding boxes suffered from severe center-of-mass mismatch due to the offset battery and ESP32 masses, yielding only a 26.7 % physical success rate. Incorporating CAD-grounded mass and spatial inertia tensors elevated success to 88.3 %.

2) Complementary Sensor Filtering: Deploying with unfiltered accelerometer tilt calculations triggered severe high-frequency servo chatter, causing gear stripping within three trials. The integrated complementary filter ($\alpha = 0.98$) eliminated high-frequency noise while maintaining phase fidelity.

5.4 Reality Gap Discussion

The residual gap between simulated (2.03 s) and real (2.36 s) righting time stems from three unmodeled real-world phenomena:

1. *Gear Backlash:* Inexpensive SG90 servos exhibit ±2.5° of backlash deadband.
2. *Transient Voltage Drop:* Instantaneous high-torque push-off events caused battery voltage dips from 7.4 V to 6.6 V, briefly degrading servo torque.
3. *Ground Contact Compliance:* Real carpet/table surfaces exhibit compliance and anisotropic friction, contrasting with MuJoCo's rigid Coulomb ground model.

Despite these unmodeled dynamics, the onboard observation structure provided sufficient feedback authority to right the robot reliably.

6 Conclusion

We presented `simtoreal`, an automated deployment framework that bridges the gap between simulation-trained reinforcement learning policies and resource-constrained microcontrollers. By synthesizing deterministic, zero-dependency C code and auto-wiring hardware abstraction layers from a single YAML file, the framework enables zero-shot sim-to-real transfer on low-cost robots. Future work will extend the generator to support recurrent architectures (LSTM/GRU) and multi-agent coordination.

References

[1] J. Hwangbo *et al.*, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[2] T. Miki *et al.*, "Learning robust perceptive locomotion for quadrupedal robots in the wild," *Science Robotics*, vol. 7, no. 62, p. eabk2822, 2022.

[3] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proc. Conf. Robot Learn. (CoRL)*, 2022, pp. 91–100.

[4] J. Tobin *et al.*, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2017, pp. 23–30.

[5] J. Tan *et al.*, "Sim-to-real: Learning agile locomotion for quadruped robots," in *Proc. Robotics: Sci. Syst. (RSS)*, 2018.

[6] M. Mittal *et al.*, "Orbit: A unified simulation framework for interactive robot learning via tensored physics," *IEEE Robot. Autom. Lett.*, vol. 8, no. 6, pp. 3740–3747, 2023.

[7] R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," in *Proc. Conf. Mach. Learn. Syst. (MLSys)*, 2021, pp. 800–811.

[8] J. Lin *et al.*, "MCUNet: Tiny deep learning on IoT devices," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 11711–11722.

[9] F. Pessia, M. Neunert, and N. Cazenave, "RL-tools: A pure C++ reinforcement learning library for resource-constrained edge devices," *arXiv preprint arXiv:2310.00035*, 2023.